

Exceptions and Error Handling

Introduction to computer programming

Management and Artificial Intelligence



Why Exceptions?

So far...

- our programs run assuming everything goes well
- but in the real world things fail: invalid input, missing files, network errors

Exceptions let us:

- detect unexpected conditions
- separate normal code from error-handling code
- recover gracefully instead of crashing

What is an Exception?

An **exception** is a signal that something went wrong during execution.

- Raised by Python or your code with `raise`
- Interrupts normal control flow until it is handled
- If unhandled, the program terminates with a traceback

Basic try/except

Use try/except to handle a specific error and keep the program running.

```
user_input = '42a' # pretend this comes from input()

try:
    number = int(user_input)
    print('Parsed number:', number)
except ValueError as e:
    print('Invalid integer:', e)
    number = None

print('number =', number)
```

```
Invalid integer: invalid literal for int() with base 10: '42a'
number = None
```

Basic try/except

Use try/except to handle a specific error and keep the program running.

```
user_input = '42a' # pretend this comes from input()

try:
    number = int(user_input)
    print('Parsed number:', number)
except ValueError as e:
    print('Invalid integer:', e)
    number = None

print('number =', number)
```

```
Invalid integer: invalid literal for int() with base 10: '42a'
number = None
```

NOTE: Catch only the exceptions you expect (e.g., `ValueError`). Catching everything with a bare `except:` can hide bugs.

Catching Multiple Exceptions

You can handle different exceptions differently, or group them in a tuple.

```
def safe_divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError as e:  
        print('Cannot divide by zero:', e)  
        return None  
    except TypeError as e:  
        print('Wrong types:', e)  
        return None  
  
print(safe_divide(10, 0))  
print(safe_divide('10', 5))  
print(safe_divide(10, 2))
```

Cannot divide by zero: division by zero

None

Wrong types: unsupported operand type(s) for /: 'str' and 'int'

None

5.0

else and finally

- `else`: runs only if no exception occurred in the `try` block
- `finally`: always runs (cleanup), regardless of exceptions

```
def read_int(text):  
    try:  
        x = int(text)  
    except ValueError:  
        print('Not an integer')  
    else:  
        print('Parsed:', x)  
        return x  
    finally:  
        print('Done attempting to parse')  
  
read_int('123')  
read_int('abc')
```

```
Parsed: 123  
Done attempting to parse  
Not an integer  
Done attempting to parse
```

Raising Exceptions

Use `raise` to signal invalid states or inputs from your own code.

```
def set_age(age):  
    if age < 0:  
        raise ValueError('age must be non-negative')  
    return age  
  
try:  
    set_age(-3)  
except ValueError as e:  
    print('Error:', e)
```

Error: age must be non-negative

Common Built-in Exceptions (selection)

- `ValueError`: right type, wrong value (e.g., `int('abc')`)
- `TypeError`: wrong type (e.g., `'3' + 4`)
- `KeyError`: missing dict key
- `IndexError`: list index out of range
- `ZeroDivisionError`: division by zero
- `FileNotFoundError`: file does not exist

Custom Exceptions

Create domain-specific exceptions by subclassing Exception.

```
class InvalidGradeError(Exception):  
    pass  
  
def record_grade(student, grade):  
    if not (0 <= grade <= 30):  
        raise InvalidGradeError(f'grade {grade} out of range 0..30')  
    return { 'student': student, 'grade': grade }  
  
try:  
    record_grade('Alice', 35)  
except InvalidGradeError as e:  
    print('Custom error:', e)
```

Custom error: grade 35 out of range 0..30

Exception Hierarchy (selected)

- Exception
 - `ArithmeticError` → `ZeroDivisionError`, `OverflowError`
 - `LookupError` → `IndexError`, `KeyError`
 - `ValueError`
 - `TypeError`
 - `FileNotFoundError` (from `OSError`)
 - ...

Catching a base class (e.g., `LookupError`) also catches its subclasses.

Exception Hierarchy (selected)

- Exception
 - `ArithmeticError` → `ZeroDivisionError`, `OverflowError`
 - `LookupError` → `IndexError`, `KeyError`
 - `ValueError`
 - `TypeError`
 - `FileNotFoundError` (from `OSError`)
 - ...

Catching a base class (e.g., `LookupError`) also catches its subclasses.

BEST PRACTICES:

- Validate inputs early; raise specific exceptions
- Catch only what you can handle; let the rest propagate
- Log exception details for debugging
- Use `finally` or context managers to release resources

Working with Files: try/finally vs Context Managers

Always close files even if an error occurs.

```
# try/finally
f = open('data.txt', 'w')
try:
    f.write('hello')
finally:
    f.close()

# context manager (preferred)
with open('data.txt', 'r') as f:
    print('Read:', f.read())
```

Read: hello

Getting the Traceback (for logging)

Use `traceback` to capture the stack trace for logs or error reports.

```
import traceback

def divide(a, b):
    return a / b

try:
    divide(3, 0)
except Exception:
    tb = traceback.format_exc()
    print('Traceback captured for logs:
', tb)
```

```
Cell In[7], line 10
    print('Traceback captured for logs:
      ^
```

SyntaxError: unterminated string literal (detected at line 10)

Putting It Together: Robust Function

Design functions that validate inputs and report errors clearly.

```
def average(values):  
    if not values:  
        raise ValueError('values must be a non-empty iterable')  
    try:  
        total = sum(values)  
        count = len(values)  
        return total / count  
    except TypeError as e:  
        raise TypeError('values must be numbers') from e  
  
try:  
    print('avg:', average([10, 20, 30]))  
    print('avg:', average([]))  
except Exception as e:  
    print('Handled:', e)
```

NOTE: Use `raise ... from e` to preserve the original cause while adding context.

Summary

- Exceptions separate error handling from normal logic
- Use specific except clauses, else, and finally
- Raise custom exceptions for domain errors
- Log tracebacks; use context managers for resources
- Favor clear error messages and input validation

